

Sarus Firmware Plan

How Sarus builds its own ArduPilot firmware for Plane (QuadPlane included), Copter and Rover, what the second release adds, how a release reaches the Install Firmware page, and what comes next.

01 Where it stands

Sarus firmware now comes in two lines. The 4.7 line lives on the branch `sarus-4.7.1` of `github.com/LikhitS/ardupilot` and starts from ArduPilot's stable 4.7.1, where Plane, Copter and Rover share one commit. The 4.6 line lives on `sarus-4.6.3` and starts from Copter 4.6.3. That commit is Plane 4.6.3 and Rover 4.6.3 plus a one-line fix to Copter's version file, so all three vehicles are built from exactly their official 4.6.3 source.

The first release, Sarus-1, changed only the name. The second, Sarus-2, is the same on both lines and adds two things. The parameter lock refuses setup changes from any ground station until it proves it holds the owner's admin password; flying stays open to everyone. The firmware also gains room for the owner's parameter set as its defaults. The aircraft identify themselves as, for example, `ArduPlane V4.7.1 Sarus-2` or `ArduCopter V4.6.3 Sarus-2`; the `Ardu...` V prefix stays because ground stations use it to tell a QuadPlane from a Copter.

GitHub's cloud machines build both lines for seven board families: CubeOrange, CubeOrangePlus, Pixhawk6X, Pixhawk6C, Pixhawk1, fmuv3 and MatekH743. One board needed an exception on the 4.6 line. ArduPilot 4.6.3 leaves CubeOrangePlus about 480 bytes of flash for Copter and 12 KB for Plane, and the lock's signature check needs about 13 KB once the owner's keys are in. On that board, at the owner's request, the 4.6.3 VTOL build (Plane) leaves out two marine and ground-vehicle features a QuadPlane never uses, Torqeedo electric boat motors and AIS ship receivers, and then fits with 1 KB to spare. Rover is built in full. Copter is not built for CubeOrangePlus on 4.6.3 until a hover build needs it; the 4.7 line has room for all three on that board.

02 The parameter lock

While locked, the aircraft refuses parameter writes and sends back the value it keeps, and it refuses parameter resets, every calibration, reboot into the bootloader, bootloader flashing, SD card formatting, receiver pairing, Lua scripting commands, OSD parameter changes, MAVLink signing setup, any file write over MAVLink FTP, log erasing, serial passthrough, device register writes, and the auxiliary functions that save setup (compass calibration, save trim, compass learning, autotune and the like). FTP writes are included because a parameter file written there loads parameters, and a script copied there can set them. Parameter menus on a FrSky or CRSF transmitter work only while a ground station holds an unlock. A plain reboot, missions, modes, arming and flying stay open.

Unlock codes mix the board's random source with packet arrival times, the boot count and the previous code, because the older F4 boards (Pixhawk1, fmuV3) have no hardware random generator in use. A station cannot use up or end another station's unlock, and after a failed signature check the next one waits a quarter of a second. Without MAVLink signing the aircraft knows a station only by its link and system ID, so links that others can reach should also use MAVLink signing. The public test key cannot be built into board firmware, and a release is refused unless every image reports the tagged version, holds the owner keys and does not hold the test key.

The aircraft holds only Ed25519 public keys: the owner's key and a spare, both created on the owner's PC on 6 October 2026. The private key behind each one is derived from its password with scrypt and exists only inside Sarus Operation Planner while someone who typed the password has it unlocked. To unlock, Sarus asks the aircraft for a one-time number and returns it signed. A number is good for one attempt, for 30 seconds, from the station and link that asked for it, so a recorded unlock cannot be replayed. The unlock belongs to that station on that link. It ends on reboot, on a lock request, or after 10 seconds of silence once disarmed; it is never cut off in flight, but a new one is refused while armed.

WHAT THE LOCK DOES NOT COVER

Someone with the board on a bench can still flash other firmware through the USB bootloader. Closing that gap needs ArduPilot's secure bootloader with a Sarus signing key, the next step below. Settings inside DroneCAN peripherals are not covered either.

03 How it was tested

TEST	WHERE	RESULT
Ed25519 check against the RFC 8032 vectors and signatures from an independent library	Cloud, every push	13 of 13
Lock on the simulator with the public test key: every refusal path, wrong keys, replayed, expired and stolen codes, other stations and links, arming, silence, reboot	Cloud, both lines	65 of 65 on each
Release build: locked from the first boot, test key and random keys refused	Cloud, both lines	6 of 6 on each
Sarus Operation Planner against the real firmware: its signatures accepted, a write that skips the program refused	This PC, 4.6.3 and 4.7.1	52 of 52 and 53 of 53
Full regression of the planner on Sarus-2 4.7.1, every QuadPlane and Rover parameter written, read back and restored	This PC	16 of 16 steps
Copter, Rover, QuadPlane alarm flight on Sarus-2 4.6.3	This PC	All passed

04 The firmware's own limits

The request was to stop the firmware overriding values the owner sets. We wrote every parameter of QuadPlane, Copter and Rover, about 3,000, to the simulator and compared what each aircraft kept. Only one is a genuine limit. Loiter acceleration (LOIT_ACC_MAX_M, Q_LOIT_ACC_MAX_M) is capped at $g \cdot \tan(\text{maximum lean angle})$, because a multirotor tilted to 30 degrees can only accelerate sideways at about 5.66 m/s^2 without losing height. Raising the cap would not make the aircraft faster; it would only make the controller ask for something it cannot do. The other differences are values the firmware measures itself (barometer ground pressure, device IDs, mission count) and parameters stored in 8 bits, which stop at 127. Changing their storage would scramble the parameters of every aircraft already flying.

So Sarus-2 removes no limit. The ground station instead shows the value the aircraft actually holds after every write, and the new airframe limits page flags values beyond what the airframe can do, without changing them.

05 The owner's parameter set as defaults

Most boards have no room for embedded defaults, so Sarus board builds now reserve 8 KB for them wherever the flash allows. On the boards built today, that is all of them except the 4.6.3 Plane build for CubeOrangePlus, which has only 1 KB left. Giving it owner defaults would mean leaving out about 8 KB more of features the aircraft does not use, a list to agree with the owner first. When the owner's file is placed in `Tools/sarus/defaults` as `plane.parm`, `copter.parm` or `rover.parm`, each build merges it into the firmware: a new board, a parameter reset or a firmware install then starts from the owner's values, and where a board ships defaults of its own, the owner's value wins. Values that belong to one aircraft (sensor calibrations, device IDs, radio calibration, barometer ground pressure) are dropped automatically, and a file too large for the firmware stops the build with a message. Every board build also proves the merge on a scratch copy. The owner's files have not been supplied yet; until they are, the firmware starts from ArduPilot's defaults.

06 From a release to the Install Firmware page

Tag sarus-v4.7.1-S2 or sarus-v4.6.3-S2	Cloud build all boards, lock tests, Windows simulators	Release files named Sarus- Plane-4.7.1-S2- CubeOrange.apj and so on	Firmware list manifest.json on sarus-manifest, one release per line
Sarus Planner reads it at start-up next to ArduPilot's list			

The firmware list is made by ArduPilot's own manifest script, unchanged, so board IDs, USB IDs and bootloader names are exactly the ones ArduPilot publishes. The list keeps one current release per type and version line, so publishing for 4.6 never removes 4.7. When a board matches several builds, Install Firmware opens the selection window Mission Planner already has. The release step refuses to publish firmware that carries no owner key, so the lock can never ship switched off.

07 Next steps

First, the owner tries the password on a simulator built with the owner keys, using `tests\try-your-password.ps1`; nobody else can, because nobody else knows it. Then both lines are tagged as Sarus-2 releases. After that the team flashes real boards and runs the Field Test Checklist, which now includes the lock, the airframe limits and the alarm.

Later steps, each with simulator evidence and a sign-off: the secure bootloader so that only Sarus-signed firmware runs; the owner's default parameter files; and more board families, up to everything ArduPilot supports, which would mean one release per vehicle type because of GitHub's size limits.

08 Decisions taken

Lines	Two, 4.7.1 and 4.6.3, with identical Sarus changes; the planner offers both
Lock	In the firmware, with Ed25519 keys from the owner's password, rather than MAVLink signing, which would also stop people without the password from flying
Keys	Owner key and a spare; public keys in the repository, passwords nowhere
Firmware limits	None removed: the only real one is physics; values the aircraft changes are shown instead
Version names	ArduPilot version plus the Sarus release, e.g. 4.6.3-S2
Repository	Public fork under LikhitS for now, to move to a company account later

09 Licensing

ArduPilot is licensed under the GPL version 3. Sarus firmware may be sold or supplied with aircraft, provided anyone who receives it can get the complete source of the build they received; the public fork does that. The lock's source is public too, and its security rests on the owner's password, not on the code being secret. Saying the firmware is based on ArduPilot is fine; suggesting ArduPilot endorses it is not.

